

# Basic

## C++ 语法基础

### 代码框架

如果你不想深究背后的原理，初学时可以直接将这个「框架」背下来：

```
1 #include <iostream>
2 #include <iostream>
3
4 int main() {
5     // do something...
6     return 0;
7 }
```

- ▶ 什么是 `include`?
- ▶ 什么是 `main()`?

### 注释

在 C++ 代码中，注释有两种写法：

#### 1. 行内注释

以 `//` 开头，行内位于其后的内容全部为注释。

#### 2. 注释块

以 `/*` 开头，`*/` 结尾，中间的内容全部为注释，可以跨行。

注释对程序运行没有影响，可以用来解释程序的意思，还可以在让某段代码不执行（但是依然保留在源文件里）。

在工程开发中，注释可以便于日后维护、他人阅读。

在 OI 中，很少有人写许多注释，但注释可以便于在写代码的时候理清思路，或者便于日后复习。而且，如果要写题解、教程的话，适量的注释可以便于读者阅读，理解代码的意图。希望各位同学能养成写注释的好习惯。

## 输入与输出

### `cin` 与 `cout`

```
1 #include <iostream>
2
3 int main() {
4     int x, y;                // 声明变量
5     std::cin >> x >> y;      // 读入 x 和 y
6     std::cout << y << std::endl << x; // 输出 y，换行，再输出 x
7     return 0;                // 结束主函数
8 }
```

- ▼ 什么是变量？

可以参考 [变量](#) 页面。

## ▼ 什么是 std?

std 是 C++ 标准库所使用的 **命名空间**。使用命名空间是为了避免重名。

关于命名空间的详细知识，可以参考 [命名空间](#) 页面。

## scanf 与 printf

scanf 与 printf 其实是 C 语言提供的函数。大多数情况下，它们的速度比 cin 和 cout 更快，并且能够方便地控制输入输出格式。

## ▼ 读入输出优化

cin/cout 和 scanf/printf 的具体差别和读入输出优化，请参考 [读入、输出优化](#) 页面。

```
1 #include <cstdio>
2
3 int main() {
4     int x, y;
5     scanf("%d%d", &x, &y);    // 读入 x 和 y
6     printf("%d\n%d", y, x);    // 输出 y，换行，再输出 x
7     return 0;
8 }
```

其中，%d 表示读入/输出的变量是一个有符号整型（int 型）的变量。

类似地：

1. %s 表示字符串。
2. %c 表示字符。
3. %lf 表示双精度浮点数 (double)。
4. %lld 表示长整型 (long long)。根据系统不同，也可能是 %I64d。
5. %u 表示无符号整型 (unsigned int)。
6. %llu 表示无符号长整型 (unsigned long long)，也可能是 %I64u。

除了类型标识符以外，还有一些控制格式的方式。许多都不常用，选取两个常用的列举如下：

1. %1d 表示长度为 1 的整型。在读入时，即使没有空格也可以逐位读入数字。在输出时，若指定的长度大于数字的位数，就会在数字前用空格填充。若指定的长度小于数字的位数，就没有效果。
2. %.6lf，用于输出，保留六位小数。

这两种运算符的相应地方都可以填入其他数字，例如 %.3lf 表示保留三位小数。

- 「双精度浮点数」，「长整型」是什么
- 为什么 scanf 中有 & 运算符？
- 什么是 \n？

## 一些扩展内容

### C++ 中的空白字符

在 C++ 中，所有空白字符（空格、制表符、换行），多个或是单个，都被视作是一样的。（当然，引号中视作字符串的一部分的不算。）

因此，你可以自由地使用任何代码风格（除了行内注释、字符串字面量与预处理命令必须在单行内），例如：

```

1 /* clang-format off */
2
3 #include <Tiostream>
4
5 int
6
7     main(){
8 int/**/x, y;  std::cin
9 >> x >>y;
10
11         std::cout <<
12         y  <<std::endl
13         << x
14         ;
15
16     return      0;      }

```

当然，这么做是不被推荐的。

一种也被广泛使用但与 **OI Wiki** 要求的码风不同的代码风格：

```

1 /* clang-format off */
2
3 #include <Tiostream>
4
5 int main()
6 {
7     int x, y;
8
9     std::cin >> x >> y;
10    std::cout << y << std::endl << x;
11
12    return 0;
13 }

```

## #define 命令

`#define` 是一种预处理命令，用于定义宏，本质上是文本替换。例如：

```

1 #include <Tiostream>
2 #define n 233
3
4 // n 不是变量，而是编译器会将代码中所有 n 文本替换为 233，但是作为标识符一部分的
5 // n 的就不会被替换，如 fn 不会被替换成 f233，同样，字符串内的也不会被替换
6
7 int main() {
8     std::cout << n;  // 输出 233
9     return 0;
10 }

```

► 什么是标识符？

► 什么是预处理命令？

宏可以带参数，带参数的宏可以像函数一样使用：

```

1 #include <LTIostream>
2 #define sum(x, y) ((x) + (y))
3 #define square(x) ((x) * (x))
4
5 int main() {
6     std::cout << sum(1, 2) << ' ' << 2 * sum(3, 5) << std::endl; // 输出 3 16
7 }

```

但是带参数的宏和函数有区别。因为宏是文本替换，所以会引发许多问题。如：

```

1 #include <LTIostream>
2 #define sum(x, y) x + y
3 // 这里应当为 #define sum(x, y) ((x) + (y))
4 #define square(x) ((x) * (x))
5
6 int main() {
7     std::cout << sum(1, 2) << ' ' << 2 * sum(3, 5) << std::endl;
8     // 输出为 3 11, 因为 #define 是文本替换, 后面的语句被替换为了 2 * 3 + 5
9     int i = 1;
10    std::cout << square(++i) << ' ' << i;
11    // 输出未定义, 因为 ++i 被执行了两遍
12    // 而同一个语句中多次修改同一个变量是未定义行为 (有例外)
13 }

```

使用 `#define` 是有风险的（由于 `#define` 作用域是整个程序，因此可能导致文本被意外地替换，需要使用 `#undef` 及时取消定义），因此应谨慎使用。较为推荐的做法是：使用 `const` 限定符声明常量，使用函数代替宏。

但是，在 OI 中，`#define` 依然有用武之处（以下两种是不被推荐的用法，会降低代码的规范性）：

1. `#define int long long+signed main()`。通常用于避免忘记开 `long long` 导致的错误，或是调试时排除忘开 `long long` 导致错误的可能性。（也可能导致增大常数甚至 TLE，或者因为爆空间而 MLE）
2. `#define For(i, l, r) for (int i = (l); i <= (r); ++i)`、`#define pb push_back`、`#define mid ((l + r) / 2)`，用于减短代码长度。

不过，`#define` 也有优点，比如结合 `#ifdef` 等预处理指令有奇效，比如：

```

1 #ifdef LINUX
2 // code for linux
3 #else
4 // code for other OS
5 #endif

```

可以在编译的时候通过 `-DLINUX` 来控制编译出的代码，而无需修改源文件。这还有一个优点：通过 `-DLINUX` 编译出的可执行文件里并没有其他操作系统的代码，那些代码在预处理的时候就已经被删除了。

`#define` 还能使用 `#`、`##` 运算符，极大地方便调试。

本页面最近更新：2023/5/6 19:22:00, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[abc1763613206](#), [billchenchina](#), [BobRocket](#), [CamberLoid](#), [CCXXXI](#), [Chrogeek](#), [Enter-tainer](#), [H-J-Granger](#), [lr1d](#), [isdanni](#), [ksyx](#), [lihaoyu1234](#), [mgt](#), [opsiff](#), [orzAtalod](#), [ouuan](#), [ouuan](#), [StudyingFather](#), [Tiphereth-A](#), [Xeonacid](#)

